```
stops = 0
while queue is not empty and stops ≤ k:
    size = len(queue)
    temp = copy of dist
    for i in range(size):
        node, cost = queue.pop(0)
        for (neighbour, price) in adj[node]:
            newCost = cost + price
            If newCost < temp[neighbour]:
                temp[neighbour] = newCost
                Queue.append((neighbour, newcost))
    dist = temp
    stops += 1
If dist[dst] == INF:
    Return -1
Return dist[dst]
```

Time complexity: $O(K \times E)$

Space complexity: $O(n+E)$

Date
Page

Experiment-7

Name- Tanay Manish Nesari
Class- 2-B
UID- 23BCS13761

Problem: There are n cities connected by some number of flights. You are given an array flights, where flights[i] = [from, to, price], which indicates that there is a flight from city 'from' to city 'to' with price 'price'. You are given 3 integers, src, dst, k, return cheapest price from src to dst with atmost k stops.

Input: n=4, flights = [(0,1,100), (1,2,100), (2,0,100), (1,3,600), (2,3,200)], src=0, dst=3, k=1.

Output: 700

Approach:
1) Convert flights into an adjacency list
2) Initialize Queue (stores (node, current cost)), dist[] (stores min. cost to reach each node)
3) Repeat 4 to 7 while queue not empty and stops ≤ k.
4) Process all nodes currently in queue
5) Use temp[] array to store updates
6) For each neighbour: If cheaper path is found → update and push into queue.
7) After level ends: copy temp → dist, stops++.
8) End

Pseudo Code

```
function cheapestFlight(n, flights, src, dst, k):
    adj = list of n empty lists
    for (u,v,price) in flights:
        adj[u].append((v,price))
    dist = [INF]*n
    dist[src] = 0
    queue = [(src,0)]
```